

Overall progress toward Turk-OS 1.0 (this phase in red)

0% → 100%

Contents

1	How to use this plan	1
1.1	The study loop	2
1.2	Time budget	2
2	Your five books and how each one is used	2
2.1	References outside the five books	3
3	Key technical decisions	3
4	The phase map	4
5	Phase summary	4
6	Timeline	5
7	What Turk-OS 1.0 looks like	5
8	Risks, and what to do when you are stuck	7
9	Master milestone tracker	8

1 How to use this plan

You know basic C, you have never written assembly, and you have never built an operating system. That is a normal starting point: the authors of **LOB** were in a similar position when they wrote it. This plan takes you from an empty folder to **Turk-OS 1.0**, a small UNIX-like operating system for 32-bit x86 PCs. It boots with GRUB, runs preemptive multitasking with separate address spaces, loads ELF programs from its own on-disk file system, and drops you into a shell with pipes and redirection. Everything runs in QEMU first. Real hardware is optional.

The route has sixteen phases, numbered 0 to 15. Each phase has its own PDF in docs/pdf/ and ends with a concrete, testable milestone. The percentages in each phase header (“0% → 5%”) estimate how much of the whole journey is behind you when that milestone is met. Phase 15 ends at 100%, and it also lists optional tracks for going further.

Every phase document follows the same layout, so you always know where to look. An *at a glance* box gives the duration, difficulty, prerequisites and result. *Why this phase* explains where it fits. *Concepts* explains the ideas you must understand before you write code. The *reading plan* lists book chapters with printed page numbers, each marked **REQUIRED**, **RECOMMENDED** or **OPTIONAL**. *Build it* breaks the

work into numbered tasks with starter snippets and a way to verify each one. The document ends with a debugging table, the milestone checklist, self-check questions, and stretch goals.

1.1 The study loop

The same loop works in every phase. Keep to it, because skipping the reading makes the coding much slower.

1. **Read the theory first**, usually **OSTEP**, then the **REQUIRED** items from the other books.
2. **Read the practical chapter** in **LOB** (when there is one) to see the concrete x86 steps.
3. **Look at a real system**. The MINIX lens box points to the matching **OSDI** section and source file.
4. **Build in small steps**. One task at a time, boot after each change, commit when it works.
5. **Verify**. Every task has a *Verify* line. A task is not finished until you have checked it.
6. **Write it down**. Keep a lab journal (`docs/journal.md`) with what you tried, what broke and why.
7. **Answer the self-check questions** without looking. If you can't, go back to the reading.

1.2 Time budget

The estimates assume about 10–12 hours a week, which adds up to roughly 43 weeks (about ten months). Some phases will go faster than planned and some much slower; paging (Phase 6) and processes (Phase 8) are where most people get stuck. What counts is meeting each phase's exit criteria. Never start a phase while the previous milestone is still shaky, because every later phase builds on top of it.

2 Your five books and how each one is used

The books in `Books/` complement each other. Only **LOB** tells you what to type. The others explain why it works and show how real systems do it. The file `Konrul_LIFT-UP_Niyet_Mektubu.pdf` in the same folder is an FPGA project application, not an OS text, so this plan does not use it.

Code	Book	Role in this plan	How to read it
LOB	The little book about OS development (78 pp.)	The practical spine. Its 14 short chapters are the build order for Phases 1–14.	Read each chapter right before its phase. It uses GRUB Legacy and Bochs; the phase docs give the modern QEMU and GRUB 2 equivalents.
OSTEP	Operating Systems: Three Easy Pieces	Main theory text. Short, friendly chapters built around one problem each.	Your first theory read in every phase. Do the homework simulators when a phase suggests it.
OSC	Operating System Concepts, 10th ed.	Encyclopedic reference. Good second explanation and exam-style depth.	Use it when an OSTEP chapter didn't click, or when you want the complete picture of a topic.
MOS	Modern Operating Systems, 4th ed.	Strongest on hardware, I/O, clocks, disks, security and OS design (Ch. 12).	Read the hardware and I/O sections closely; they explain the devices you will program.
OSDI	OS: Design and Implementation, 3rd ed.	The only book with a complete real x86 kernel: MINIX 3, printed line by line.	After you understand a topic, read how MINIX implements it. Don't copy it: MINIX is a microkernel, Turk-OS is monolithic.

Table 1: Roles of the five books.

2.1 References outside the five books

Books explain ideas, but writing a kernel also needs exact hardware facts. Keep these four free references at hand, and note that **LOB** relies on them too. The **Intel 64 and IA-32 Architectures Software Developer's Manual, Volume 3A** (System Programming Guide) is the definitive source for protection, segmentation, paging, interrupts and the TSS; you will read chapters 2–7 in pieces. The **OSDev Wiki** (<https://wiki.osdev.org>) has practical pages on nearly every device in this plan, plus lists of common mistakes. The **Multiboot Specification 0.6.96** defines the contract between GRUB and your kernel. The **NASM manual** covers assembler syntax. The MIT **xv6** teaching kernel is a good optional companion: it is small and UNIX-like, and **OSTEP** refers to it.

3 Key technical decisions

These choices hold for the whole plan. Each one follows from the books you have, which keeps your reading and your code in step.

Decision	Choice	Why
Target CPU	IA-32 (i386), 32-bit protected mode, BIOS boot	LOB , OSDI (MINIX 3) and the Intel examples in OSC §9.6 are all 32-bit x86. Two-level paging is the simplest real MMU to learn. A 64-bit port is a Phase 15 track.
Boot loader	GRUB via Multiboot 1; qemu -kernel for quick runs	Writing a boot loader is a detour. You write one 512-byte boot sector in Phase 1 to learn how booting works, then let GRUB do the job.
Languages	C (gnu11) plus NASM (Intel syntax)	Assembly only where C cannot work: the entry point, descriptor table loads, interrupt stubs, context switch and entering user mode. That is under 5 % of the code.
Toolchain	i686-elf cross GCC and binutils, GNU Make, Git	A cross compiler never assumes a Linux target, which prevents a family of confusing bugs (Phase 0).
Emulator, debugger	QEMU and GDB (remote stub); Bochs is optional	QEMU is fast, scriptable, and has a monitor for inspecting registers and page tables. LOB uses Bochs, whose debugger is still handy for descriptor-table bugs.
Kernel structure	Monolithic, modular source tree	Simplest to get right for a first kernel. MINIX's microkernel is studied as a contrast, with an optional experiment in Phase 15.
On-disk file system	MINIX V3 layout with 1 KiB blocks, named TurkFS	The format described in OSDI §5.6, with source. Linux's <code>mkfs.minix -3</code> and <code>fsck.minix</code> create and check your images, and Linux can even mount them.
Testing	In-kernel ktest suites, headless QEMU, isa-debug-exit	make test boots, runs every test and exits with a pass or fail code, so regressions show up at once (Phase 7).

Table 2: Technical decisions for Turk-OS.

Why monolithic, when one of your books is about a microkernel?

In a monolithic kernel, drivers, the file system and the scheduler all run in ring 0 and call each other as plain C functions. In a microkernel like MINIX 3 they run as separate user-space processes that talk by message passing (**OSDI** §1.5.5, §2.5.1; **MOS** §1.7). Microkernels isolate faults better, but they force you to build IPC, servers and a protocol for every operation before anything works. For a first kernel, plain function calls keep the path from idea to running code short. You will still read the MINIX code in every phase, and noticing where the two designs differ is one of the best ways to understand both.

4 The phase map

The sixteen phases fall into four stages. Arrows show the build order. Each phase depends on everything before it.

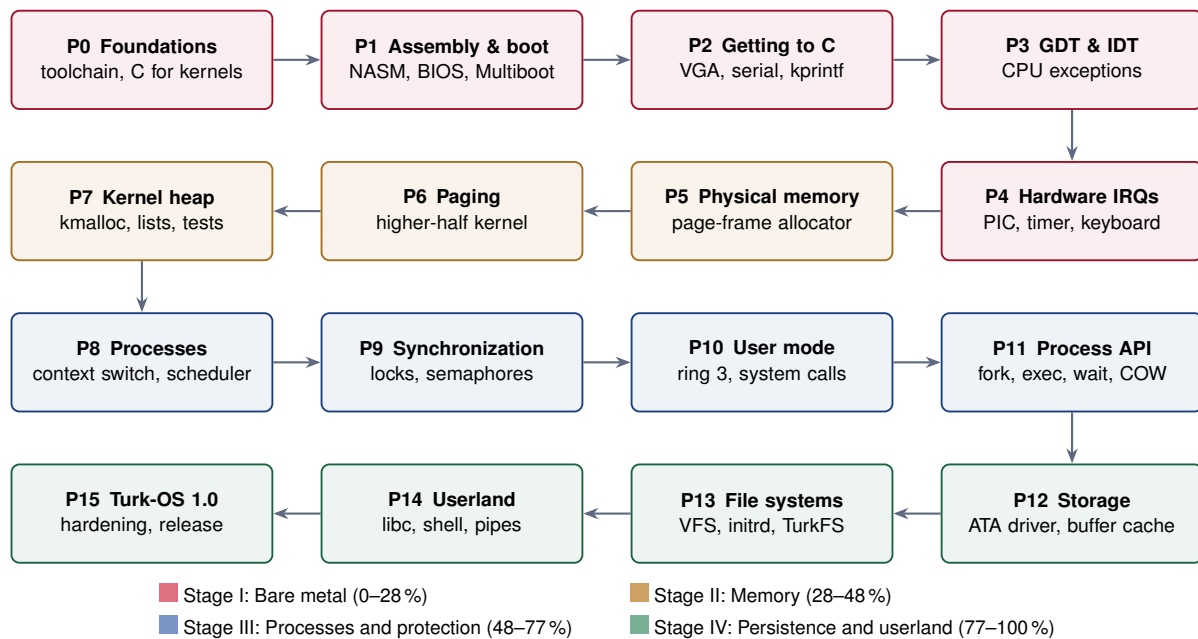


Figure 1: The sixteen phases, grouped into four stages.

Stage I, Bare metal (Phases 0–4). You learn the tools, enough assembly to be dangerous, and how a PC boots. By the end the kernel runs C code, prints to the screen and the serial port, survives CPU exceptions with a readable error report, and reacts to the timer and the keyboard through a small built-in command monitor.

Stage II, Memory (Phases 5–7). The kernel learns which RAM exists and hands it out page by page, turns on paging, moves itself to the top gigabyte of the address space, and gets a `kmallocc` heap. You also set up the automated test harness that protects you from regressions for the rest of the project.

Stage III, Processes and protection (Phases 8–11). Multitasking arrives: kernel threads, context switches, a preemptive scheduler, then the locks that make concurrency safe. Programs move to ring 3 with their own address spaces and talk to the kernel through system calls. The stage ends with the UNIX process model: `fork` with copy-on-write, `exec`, `wait` and `exit`.

Stage IV, Persistence and userland (Phases 12–15). A disk driver and buffer cache give you storage, a VFS and TurkFS give you files, and a C library, shell and utilities give you a usable system. The last phase hardens, measures and documents it, tags version 1.0, and lays out the optional tracks.

5 Phase summary

Phase	Title	You finish with	Weeks	Progress
0	Foundations and toolchain	Cross compiler, QEMU, GDB and a Git repo; C warm-up library with unit tests	2–3	0 % → 5 %
1	x86 assembly and booting	A 512-byte boot sector; a Multiboot kernel that GRUB and QEMU boot	2–3	5 % → 10 %

Phase	Title	You finish with	Weeks	Progress
2	Getting to C: screen, serial, kprintf	VGA text driver, serial logging, kprintf, panic	1–2	10 % → 16 %
3	Segmentation, IDT and exceptions	Your own GDT and IDT; register dump on every CPU exception	2	16 % → 22 %
4	Hardware interrupts	PIC, 100 Hz timer, keyboard driver, kernel command monitor	2	22 % → 28 %
5	Physical memory management	Memory map parsing, bitmap page-frame allocator, first kernel tests	1–2	28 % → 34 %
6	Paging and the higher-half kernel	Paging on, kernel at 0xC0000000, VMM API, page-fault reports	2–3	34 % → 42 %
7	Kernel heap and data structures	kmalloc/kfree, intrusive lists, make test in headless QEMU	1–2	42 % → 48 %
8	Processes and scheduling	Kernel threads, context switch, preemptive round-robin, sleep	3	48 % → 56 %
9	Synchronization	Spinlocks, wait queues, mutexes, semaphores; blocking keyboard input	2	56 % → 62 %
10	User mode and system calls	TSS, ring 3, int 0x80, ELF loader, first user programs	3	62 % → 70 %
11	Process API	fork (copy-on-write), exec, wait, exit, brk, basic signals	3	70 % → 77 %
12	Storage	ATA PIO driver, block layer, MBR partitions, buffer cache	2	77 % → 83 %
13	File systems	VFS, file descriptors, tar initrd, writable TurkFS	3–4	83 % → 90 %
14	Userland	Turk-libc, pipes, tty line discipline, tsh shell, coreutils	3	90 % → 96 %
15	Hardening and release	Fuzzing, permissions, benchmarks, docs, v1.0 ISO; optional tracks	3–4+	96 % → 100 %

6 Timeline

The bars in Figure 2 place each phase on a calendar at 10–12 hours a week. Stage colors match the phase map. Expect Phases 6, 8 and 13 to run over; that is normal, and the rest of the schedule usually absorbs it.

7 What Turk-OS 1.0 looks like

Figure 3 shows the finished system and the phase that builds each part. It is worth coming back to this picture whenever you lose track of where a piece fits.

The source tree grows into the layout below. You don't need to create it all on day one; each phase document says which folders it adds.

Turk-OS/ (final layout)	
Turk-OS/	
-- Books/	the five reference books
-- docs/	this study plan (LaTeX), journal.md, architecture notes
-- boot/	loader.s (Multiboot entry), grub.cfg
-- kernel/	

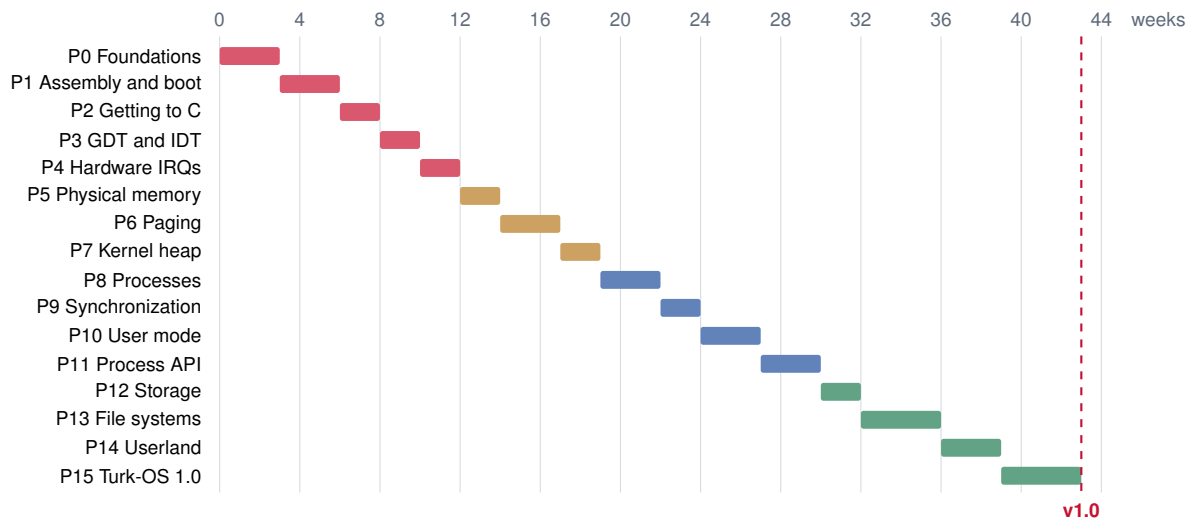


Figure 2: Planned timeline at 10–12 hours per week. Treat it as a guide, not a deadline.

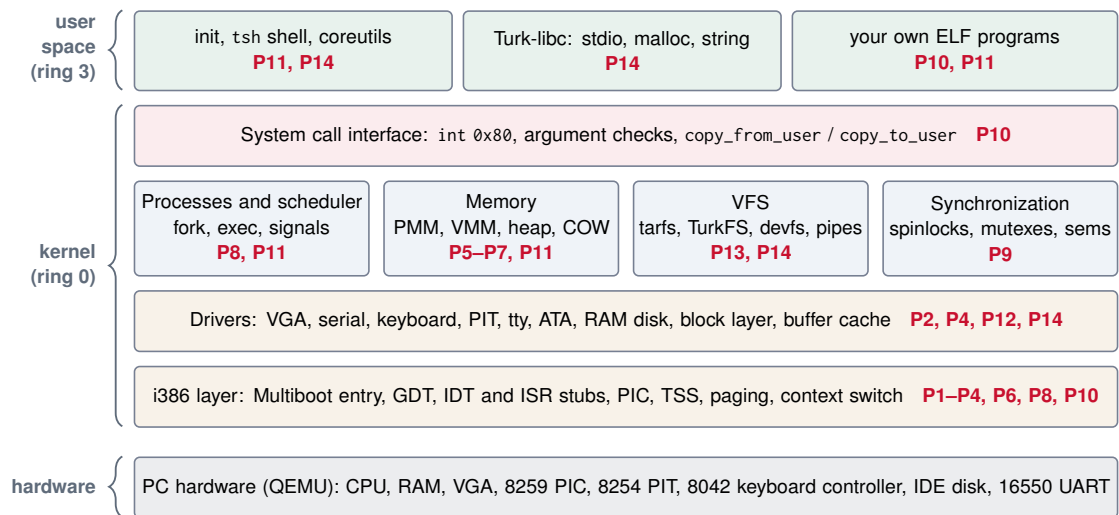


Figure 3: Layers of Turk-OS 1.0, with the phase that builds each part.

```

| |-- arch/i386/      gdt.c idt.c isr.s irq.c pic.c pit.c tss.c paging.c switch.s
| |-- mm/            pmm.c vmm.c heap.c
| |-- proc/          task.c sched.c fork.c exec.c signal.c
| |-- sync/          spinlock.c mutex.c semaphore.c waitq.c
| |-- fs/            vfs.c file.c tarfs.c turkfs.c devfs.c pipe.c
| |-- drivers/       vga.c serial.c keyboard.c tty.c ata.c ramdisk.c blockdev.c bcache.c
| |-- syscall/       syscall.c uaccess.c
| `-- kmain.c
|-- lib/             libk: string.c printf.c bitmap.c ringbuf.c list.h
|-- include/         kernel headers
|-- user/
| |-- libc/          Turk-libc
| |-- init/ sh/ bin/  init, tsh, coreutils
| `-- user.ld
|-- tests/           ktest suites
|-- tools/           mkturkfs.c and helper scripts
|-- link.ld
`-- Makefile

```

8 Risks, and what to do when you are stuck

Everyone who writes a kernel hits days where the machine silently reboots and nothing makes sense. The table lists the predictable risks. The protocol after it is how you get unstuck without losing a week.

Risk	Likelihood	Mitigation
Triple faults (instant reboot) with no output	High	Serial logging from Phase 2 on; run QEMU with <code>-d int,cpu_reset -no-reboot</code> ; GDB; small commits so git bisect can find the change that broke it.
Assembly learning curve	High	Phase 1 is dedicated to it. Read compiler output instead of writing assembly from scratch, and keep assembly to tiny stubs.
Memory corruption that shows up far from its cause	High	Assertions everywhere, magic numbers in headers, guard pages, <code>heap_check()</code> , deterministic tests.
Stuck on one bug for days	High	Use the protocol below. Time-box the bug, then change approach.
Scope creep (GUI or networking too early)	Medium	Exit criteria are the definition of done. New ideas go on a “later” list for Phase 15.
Toolchain trouble (cross-GCC build, Flatpak sandbox)	Medium	Phase 0 checklist, a host terminal, documented fallback compiler flags.
Outdated instructions in LOB	High	The phase docs give current equivalents (<code>grub2-mkrescue</code> , QEMU); the OSDev Wiki has errata.
Losing motivation	Medium	A visible milestone every 2–3 weeks. Record a short demo at each milestone and keep the journal.

💡 The getting-unstuck protocol

1. **Reproduce it reliably**, then make it smaller: comment code out until you find the shortest path to the bug.
2. **Look, don’t guess.** Use the serial log, `-d int`, the QEMU monitor (`info registers`, `info mem`), and GDB breakpoints.

3. **Write the hypothesis down** in your journal before you change code. Change one thing at a time.
4. **Re-read the section** of the Intel manual or the book chapter for the exact hardware rule involved.
5. **Compare with MINIX** (**OSDI** source) or xv6: how does a working kernel do the same step?
6. **After two sessions without progress**, ask for help (OSDev forum, a classmate, your advisor) and include your journal notes. Explaining the problem often solves it.

9 Master milestone tracker

Print this page and tick each box when that phase's exit criteria are fully met.

- ☐ **P0** i686-elf-gcc, NASM, QEMU and GDB work; repo created; C warm-up library passes its tests.
- ☐ **P1** Boot sector prints a message; Multiboot kernel boots via `-kernel` and via a GRUB ISO.
- ☐ **P2** Colored banner on screen, same log on the serial port, `kprintf` and `panic` tested.
- ☐ **P3** Own GDT and IDT loaded; divide-by-zero shows a register dump; `int3` returns cleanly.
- ☐ **P4** Timer ticks at 100 Hz; keyboard input with shift and caps; monitor commands run.
- ☐ **P5** Memory map printed; frame allocator passes its tests from 16 MiB to 3 GiB of RAM.
- ☐ **P6** Kernel runs at `0xC0000000`; null dereference produces a clean page-fault report.
- ☐ **P7** 10 000-operation heap stress test passes; `make test` runs headless.
- ☐ **P8** Several kernel threads preempt each other; `sleep` works; no leaks over 1000 task cycles.
- ☐ **P9** Race demo fixed by locks; bounded buffer and dining philosophers run without deadlock.
- ☐ **P10** Ring-3 program prints through `write`; faulty programs are killed, the kernel survives.
- ☐ **P11** `fork/exec/wait` chain works; copy-on-write verified; no leaks over 1000 forks.
- ☐ **P12** QEMU disk identified; data survives reboot; partitions detected; buffer cache in use.
- ☐ **P13** Files written by Turk-OS pass `fsck.minix` on the host; file descriptor semantics tested.
- ☐ **P14** Boots to the `tsh` prompt; pipes, redirection and `Ctrl+C` work; `coreutils` present.
- ☐ **P15** One hour of fuzzing without a panic; benchmarks and docs written; v1.0 tagged.

Books used in this plan

- LOB** Erik Helin, Adam Renberg. *The little book about OS development*, 2015. The hands-on x86 guide (Books/book.pdf).
- OSTEP** Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*, v0.90. Main theory text.
- OSC** A. Silberschatz, P.B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed., Wiley, 2018. Reference theory text.
- MOS** Andrew S. Tanenbaum, Herbert Bos. *Modern Operating Systems*, 4th ed., Pearson, 2015. Hardware, I/O, design, security.
- OSDI** Andrew S. Tanenbaum, Albert S. Woodhull. *Operating Systems: Design and Implementation*, 3rd ed., 2006. MINIX 3 with full source listing.

Page numbers are the *printed* page numbers from each book's table of contents, not PDF page indices. MINIX source references like `kernel/proc.c (07400)` give the file and its starting line in **OSDI** Appendix B.